5-Month AI Engineer Master Plan — UPDATED

AI / GenAI + Backend + Platform / Cloud + System Design

Start: 15 August 2026 • 20 weeks • Interview-ready by Months 4–5

Major update: System Design / Architecture is now a continuous weekly track from Week 1. DSA remains continuous and pattern-focused. Every major project now requires an architecture diagram, explicit trade-offs and an Architecture Decision Record (ADR). AI-specific architecture is introduced through RAG, LLM gateways, agents, MCP, multi-tenancy and security.

North Star

Design, build, deploy, secure, evaluate, observe, debug and explain production-grade GenAI systems on cloud infrastructure—not simply complete courses.

Target Role Mix

- AI / GenAI Engineer
- AI Platform Engineer
- Cloud / Platform Engineer with AI
- Backend Engineer building AI-powered systems
- AI Security / Governance as a secondary specialization

Weekly Time Budget

Day	Plan	Time
Mon (WFH)	Deep learning + DSA/revision + architecture drill	1.5–2 h
Tue (Office)	Maintenance / easy DSA / architecture flash review	≤30 min
Wed (Office)	AI/Python/system-design revision	≤30 min
Thu	Deep learning if WFH; maintenance if office	0.5–1.75 h
Fri (WFH)	Deep learning + project implementation	1.5–2 h
Sat	Main build/project + DSA + architecture diagram	4–4.5 h
Sun	Project + system design + DSA + weekly review	3–4 h

Office-day principle: sleep and recovery win. WFH days are primary learning days; weekends are primary build days.

5 Parallel Tracks

- AI / GenAI: LLMs → RAG → evaluation → agents → MCP → AI security/governance.
- Backend: Python → async/concurrency → FastAPI → testing → production API design.
- Platform / Cloud: AWS → Docker → Kubernetes/EKS → Terraform → CI/CD → observability.
- DSA / SQL: continuous patterns from Week 1; SQL becomes interview-ready by Week 17.
- System Design / Architecture: fundamentals → distributed systems → cloud architecture → AI architecture → timed interviews.

20-Week Detailed Roadmap

Week 1 — Production Python

AI / Backend	async/await, asyncio, coroutine vs task, event loop, blocking I/O; exceptions, typing, dataclasses, context managers, logging.
Platform / Cloud	Linux/process/thread basics; Git workflow; Docker mental model.
DSA / SQL	Arrays
System Design	Architecture basics: HTTP, DNS, reverse proxy, load balancer, stateless service. Draw a FastAPI architecture.
Deliverable	Python engineering lab + architecture diagram.

Week 2 — FastAPI + Backend

AI / Backend	HTTP/REST, FastAPI, Pydantic, DI, middleware, async APIs, validation, auth, pagination, pytest, mocking.
Platform / Cloud	Docker basics; Git branching/PR hygiene.
DSA / SQL	Strings
System Design	API design: resources, idempotency, authentication vs authorization, rate limiting, service boundaries.
Deliverable	Backend API + API architecture diagram.

Week 3 — LLM Fundamentals

AI / Backend	Tokens, context, embeddings, transformer/attention intuition, inference, temperature/top-p, hallucination, structured output, tool calling, streaming, cost/latency.
Platform / Cloud	AWS AI-service landscape; IAM; CloudWatch logging.
DSA / SQL	Hash Maps / Sets
System Design	Caching vs DB; synchronous request vs async job; basic latency budgeting.
Deliverable	LLM experiments + LLM API architecture.

Week 4 — Project 1 — Production LLM API

AI / Backend	Auth, chat, streaming, structured output, history, retries, timeouts, errors, logging, token/cost tracking, tests.
Platform / Cloud	Dockerize; env/secrets separation; basic CI.
DSA / SQL	Two Pointers
System Design	Design API gateway/LB, app tier, DB, cache, LLM provider and observability; explain trade-offs.
Deliverable	Production LLM API + architecture + ADRs.

Week 5 — Embeddings + Vector Search

AI / Backend	Embeddings, cosine similarity, chunking, metadata, vector indexes/DBs; implement retrieval before frameworks.
Platform / Cloud	S3/DynamoDB/relational DB; VPC review.
DSA / SQL	Fixed Sliding Window
System Design	SQL vs NoSQL, indexes, access patterns, object storage and metadata modeling.
Deliverable	Vector-search prototype + data-flow diagram.

Week 6 — Core RAG

AI / Backend	Parsing, cleaning, chunking, embeddings, indexing, retrieval, context, prompts, citations.
Platform / Cloud	AWS RAG deployment options; S3 ingestion; IAM.
DSA / SQL	Dynamic Sliding Window
System Design	Replication, consistency intuition, transactions, retries, idempotency and failure modes.
Deliverable	RAG prototype + ingestion/query architecture.

Week 7 — Advanced RAG + Evaluation

AI / Backend	Filtering, hybrid search, reranking, query rewriting, permissions, caching; precision/recall/MRR/hit-rate, faithfulness/relevance/groundedness.
Platform / Cloud	Structured logs, metrics, traces; CloudWatch.
DSA / SQL	Stacks / Queues
System Design	Cache-aside, read replicas, queues, async workers, backpressure, latency/consistency trade-offs.
Deliverable	RAG evaluation harness + measured improvement.

Week 8 — Project 2 — Enterprise RAG

AI / Backend	PDF ingestion, vector search, citations, permissions, evaluation, caching, logging, tests.

Platform / Cloud	Docker + CI/CD; secure configuration.
DSA / SQL	Linked Lists
System Design	Ingestion plane vs query plane, tenant isolation, failure handling, cost/latency budget.
Deliverable	Enterprise RAG Platform + architecture + ADR.

Week 9 — Docker + AWS

AI / Backend	Production packaging for AI service.
Platform / Cloud	Dockerfiles, networking, Compose, multi-stage builds; VPC, ALB, ECR, ECS, IAM, CloudWatch, Secrets Manager.
DSA / SQL	Dockerfiles, networking, Compose, multi-stage builds; VPC, ALB, ECR, ECS, IAM, CloudWatch, Secrets Manager.
System Design	AWS cloud architecture: Route53, ALB, compute, DB, S3, secrets, observability.
Deliverable	Deploy Project 2 + AWS architecture diagram.

Week 10 — Kubernetes

AI / Backend	Graceful shutdown, concurrency limits, readiness/liveness, resource-aware AI service.
Platform / Cloud	Pods, deployments, services, ingress, ConfigMap, Secret, namespaces, probes, HPA, rolling deployments, EKS.
DSA / SQL	Pods, deployments, services, ingress, ConfigMap, Secret, namespaces, probes, HPA, rolling deployments, EKS.
System Design	Kubernetes architecture, service discovery, scaling, failure recovery.
Deliverable	AI service on Kubernetes + deployment architecture.

Week 11 — Terraform + CI/CD

AI / Backend	AI release discipline: tests/evals, model/config versioning.
Platform / Cloud	Terraform modules/state/environments/IAM; lint → tests → security scan → Docker → registry → deploy → smoke test.
DSA / SQL	Terraform modules/state/environments/IAM; lint → tests → security scan → Docker → registry → deploy → smoke test.
System Design	IaC and deployment architecture; rollback, blue/green vs rolling, secrets.
Deliverable	Reusable infrastructure + pipeline + diagram.

Week 12 — AI Platform Engineering

AI / Backend	LLM gateway, model routing, rate limits, quotas, caching, retries, fallbacks, cost, secrets, multi-tenancy, abstraction, observability.
Platform / Cloud	AWS platform services; Kubernetes ingress/services; telemetry.
DSA / SQL	AWS platform + Kubernetes ingress/service patterns; centralized telemetry.
System Design	LLM Gateway architecture: policy, routing, provider abstraction, quotas, cache, fallback, audit.
Deliverable	LLM Gateway prototype + ADR.

Week 13 — Agents

AI / Backend	Tool calling, planning, memory, state, orchestration, workflows vs agents, human-in-loop.
Platform / Cloud	Least privilege, secrets and audit logs.
DSA / SQL	Secure tool execution, least privilege, secrets, audit logs.
System Design	Agent architecture: orchestrator, model access, tools, knowledge, memory, state, guardrails, observability.
Deliverable	Tool-using agent + architecture diagram.

Week 14 — MCP

AI / Backend	Client/server, tools/resources/prompts, transport, discovery, auth/security, MCP vs APIs/function calling.
Platform / Cloud	Read-only AWS MCP server; permission boundaries.
DSA / SQL	Build read-only AWS-oriented MCP server; safe credentials and permission boundaries.
System Design	MCP boundary, discovery, authorization, auditability, failure isolation.
Deliverable	MCP server + threat model + architecture diagram.

Week 15 — Project 3 — Agent/MCP Platform

AI / Backend	Combine orchestration, RAG, MCP, memory, tools, guardrails, evaluation, observability.
Platform / Cloud	AWS/K8s/Terraform + CI/CD.
DSA / SQL	AWS/Kubernetes/Terraform + CI/CD.
System Design	Enterprise architecture: application → agent layer → model/tool/knowledge services → data; multi-tenancy and reliability/cost trade-offs.
Deliverable	Flagship AI Agent/MCP Platform + architecture README.

Week 16 — AI Security + Governance

AI / Backend	Prompt/indirect injection, jailbreaks, data leakage, excessive agency, tool poisoning, authorization, tenant isolation, privacy, auditability; NIST AI RMF + OWASP awareness.
Platform / Cloud	IAM, isolation, secrets, logging, incident response.
DSA / SQL	IAM least privilege, isolation, secrets, logging, incident response.
System Design	Threat-model trust boundaries, assets, threats, controls and audit trail.
Deliverable	Security/governance layer + threat model + control matrix.

Week 17 — Python + DSA + SQL Revision

AI / Backend	Python internals, async, threads/processes, GIL, memory, OOP, exceptions, typing; AI/RAG rapid revision.
Platform / Cloud	Cloud/K8s/Terraform/CI-CD troubleshooting.
DSA / SQL	AWS/K8s/Terraform/CI-CD troubleshooting.
System Design	General system-design drills: requirements, estimation, API/data, scaling, reliability, security, observability.
Deliverable	Interview problem set + 2 timed architecture drills.

Week 18 — System Design Deep Dive

AI / Backend	RAG, LLM gateway, agent and multi-tenant AI platform; latency, cost, evaluation, safety.
Platform / Cloud	AWS Well-Architected review.
DSA / SQL	AWS Well-Architected review of portfolio systems.
System Design	Full framework: requirements → estimates → APIs → data → architecture → scale → reliability → security → observability → cost → trade-offs.
Deliverable	4 detailed designs + diagrams + ADRs.

Week 19 — Interview Simulations

AI / Backend	Python, AI/LLM, RAG, agents, MCP, security, project deep dives.
Platform / Cloud	AWS/K8s/Terraform troubleshooting.
DSA / SQL	AWS, Docker, Kubernetes, Terraform, CI/CD troubleshooting.
System Design	1 general + 1 AI system-design mock; draw and defend trade-offs.
Deliverable	Mock loop + gap list.

Week 20 — Application Sprint

AI / Backend	Minimal new learning; project/AI revision.
Platform / Cloud	Deployment/cloud rapid revision.
DSA / SQL	Minimal new learning; deployment/cloud revision.
System Design	2 architecture mocks + rapid review of diagrams/ADRs.
Deliverable	Applications, referrals, recruiter outreach; full interview mode.

DSA + SQL + System Design Retention

DSA Target

~140–170 high-quality problems; prioritize pattern recognition over problem-count inflation. Use spaced repetition and timed practice.

- Saturday/Sunday: 45–60 min for the week's pattern + 1–2 new problems.
- Monday: 30–45 min implementation/review.
- Tuesday office: mental walkthrough of one solved problem.
- Every 7 and 30 days: review the pattern, then solve a new problem using it.
- From Week 17: timed mixed practice and verbal explanation.

System Design Progression

- Weeks 1–4: HTTP, DNS, APIs, load balancing, stateless services, caching, queues, auth, rate limiting.
- Weeks 5–8: SQL/NoSQL, indexes, transactions, replication, consistency, partitioning/sharding, idempotency, retries, event-driven architecture.
- Weeks 9–12: AWS architecture, Kubernetes architecture, deployment strategies and platform design.
- Weeks 13–16: LLM gateways, RAG, agents, MCP, multi-tenancy, tool authorization, evaluation and AI security architecture.
- Weeks 17–20: timed designs, estimation, project deep dives and mock interviews.

System Design Interview Framework

Clarify requirements → functional/non-functional requirements → rough scale estimates → APIs → data model → high-level architecture → bottlenecks → reliability/failure handling → security → observability → cost → trade-offs/alternatives.

AI System Design Additions

- Latency: retrieval, reranking, inference, tool calls and network overhead.
- Cost: tokens, embeddings, vector storage, tool calls and retries.
- Quality: retrieval metrics, groundedness, relevance and evaluation gates.
- Security: prompt/indirect injection, data leakage, excessive agency, tenant isolation and tool authorization.
- Production controls: routing, quotas, caching, fallbacks, audit logs, tracing and human approval for sensitive actions.

Architecture Reference Library

AWS Architecture Center

Reference architectures, diagrams and decision guidance.

https://aws.amazon.com/architecture/

AWS Well-Architected Framework

Architecture evaluation across operational excellence, security, reliability, performance efficiency, cost optimization and sustainability.

https://docs.aws.amazon.com/wellarchitected/latest/framework/

AWS RAG Architecture Guidance

Options and trade-offs for managed and custom RAG architectures.

https://docs.aws.amazon.com/prescriptive-guidance/latest/retrieval-augmented-generation-options/introduction.html

AWS Enterprise Agentic AI Architecture

Layered enterprise architecture for applications, agents, model access, tools, knowledge bases, security and observability.

https://docs.aws.amazon.com/prescriptive-guidance/latest/govern-architect-agentic-ai/enterprise-architecture.html

Designing Data-Intensive Applications — Martin Kleppmann

Distributed-data principles: scalability, consistency, reliability, fault tolerance and technology trade-offs. Prefer the current 2nd edition where accessible.

https://martin.kleppmann.com/

ByteByteGo System Design Cheat Sheet

15 core areas including requirements, architecture, data, scalability, reliability, security, cost and migration.

https://bytebytego.com/guides/a-cheat-sheet-for-system-designs/

OpenAI Cookbook — Agents

Practical agent engineering patterns.

https://developers.openai.com/cookbook/topic/agents

OpenAI Cookbook — Evals

Evaluation patterns for measuring and improving AI systems.

https://developers.openai.com/cookbook/topic/evals

Three Core Portfolio Projects

- Project 1 — Production LLM API: Python, FastAPI, LLM API, PostgreSQL, Docker, AWS, Terraform, CI/CD, CloudWatch, IAM.
- Project 2 — Enterprise RAG Platform: Python, FastAPI, embeddings, vector DB, RAG, evaluation, AWS, Docker, Terraform, CI/CD, security, observability.
- Project 3 — Enterprise AI Agent / MCP Platform: orchestration, RAG, MCP, tools, memory, guardrails, evaluation, observability, AWS/Kubernetes/Terraform.

Project Architecture Definition of Done

- Architecture diagram showing request/data/control flow.
- API and data model documented.
- At least 5 explicit trade-offs with rejected alternatives.
- Failure modes and recovery strategy.
- Security boundaries, authorization and secrets strategy.
- Logs, metrics, traces and useful alerts.
- Cost and latency considerations.
- ADR for each major design choice.
- Explain the entire design without notes.

Future-Proofing Rules

- Frameworks change; fundamentals should remain.
- Do not chase every new model, agent framework, vector database or protocol.
- Use MCP as an integration/protocol skill, not an isolated specialization.
- Prefer measured evaluation over demos.
- Treat security/governance as architecture concerns.
- Keep DSA and system design running in parallel.

Week-20 Target Profile

Area	Target	Evidence
Python	4/5	Projects + interview answers
AWS	4/5	Deployed systems + architecture
FastAPI	3.5/5	Production API
LLM / RAG	4/5	Projects + evaluation
Agents	3.5/5	Agent platform
MCP	3/5	Working server/client integration
Docker	3.5/5	Containerized deployments
Kubernetes	3/5	Practical deployment + EKS

Area	Target	Evidence
Terraform	3.5/5	Reusable infrastructure
CI/CD	3/5	Working pipeline
System Design	3.5/5	AI + general designs + trade-offs
AI Security	3/5	Threat model + controls
AI Governance	3/5	Governance concepts + controls
DSA	3.5/5	Pattern-based readiness

Interview Readiness Checklist

- Explain Python async/concurrency and debug blocking I/O.
- Build and defend a FastAPI service.
- Explain LLM fundamentals, latency and cost.
- Design and evaluate RAG.
- Design a tool-using agent and explain workflows vs agents.
- Explain MCP and build a basic integration.
- Threat-model prompt injection, excessive agency, data leakage and tenant isolation.
- Deploy a containerized AI service on AWS.
- Explain Docker/Kubernetes/EKS.
- Write and defend Terraform and CI/CD.
- Design an LLM gateway and multi-tenant AI platform.
- Solve common DSA patterns and SQL problems.
- Perform AI and general system-design interviews with explicit trade-offs.
- Deep-dive into all three portfolio projects without notes.

North Star: Build → Break → Debug → Explain → Design → Interview